CUDA Course

Victor Eijkhout

2025 COE 379L / CSE 392

Justification

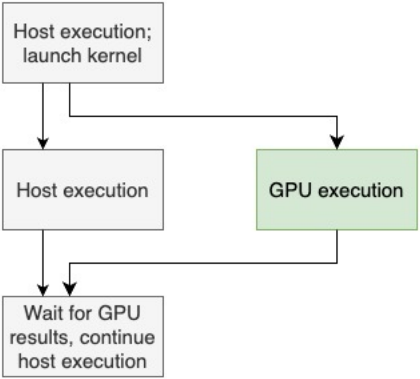
CUDA is the basic programming model for NVidia GPUs ...
... which TACC is getting a lot of.

1. Introduction

Why GPUs

- Massive parallelism (but that's really multicore and SIMD)
- Simpler processor: less power / more computing for the same power.
- Simpler processor: no general programs, only 'data parallel' operations
  ... such as machine learning
- Latest GPUs have very high speed on low precision
  ... such as for machine learning

Attached processor



- GPU is attached to regular CPU
- computation is 'offloaded'
- GPU has its own memory:
  you may need to synchronize

Data parallel programming

- Kernel: operation for a single thread;
  in effect: body of inner loop
- Grid: multi-dimensional structure of available 'threads'
  executed on cores
  (not a CPU core, more SIMD vector lane)

Processor structure

- Streaming Multiprocessor: like a core
- Warp: vector instruction
- Thread: like a simd-lane
- Single Instruction Multiple Thread (SIMT) 'massive multi-threading': you can have way more 'threads' than $\mathrm{blocks} \times \mathrm{blocksize}$ processor can switch very fast between threads

Loops vs CUDA

```
float *x;
for ( blockIdx_x < gridsize )
  for ( threadIdx_x < blocksize )
    size_t linear = threadIdx_x
      + blockIdx_x * blocksize;
    f( x[linear] );
```

```
__global__ cu_f( float x) {
  size_t linear = threadIdx_x
      + blockIdx_x * blocksize;
  f( x[linear] );
}

float *x; // on device
cu_f<<<gridsize,blocksize>>>(x);
```

GPU memory

- Very much graphics inspired:
  'constant' memory, 'texture' memory
- Main memory: has grown from 8G to 100G-ish
- Shared memory inside each SM: managed cache
- Actual caches and registers.

Comparison

- Dozens of cores
- Core has vector instructions
- a vector instruction has ‘lanes’
- Latency hiding by caches, prefetching

- Dozens of ‘Streaming Multiprocessors’
- SM has ‘warp’s
- a warp has threads
- Latency hiding by massive multi-threading

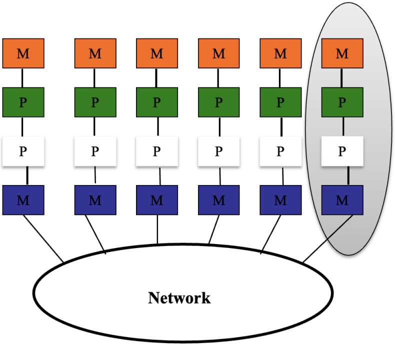
GPU cluster

Memory GPU

GPU

CPU

Memory CPU



Step 1:
One node

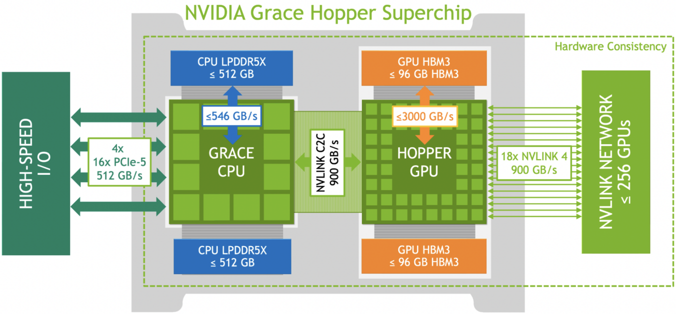
Step 2:
Multiple nodes

- Use GPU on a node
  (possible combined with OpenMP or other threading)
- Use MPI between nodes;
  NVlink also exists but not on large scale

TACC's Vista

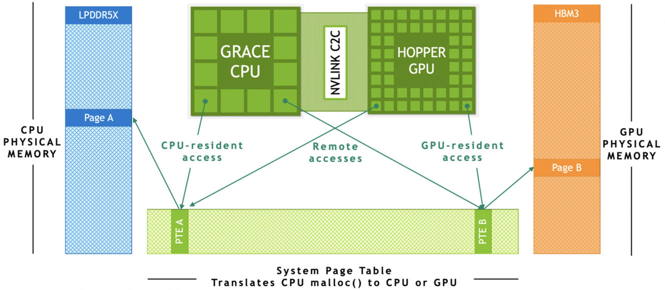
- 256 Grace-Grace two-socket CPU nodes
  4 racks of 64 nodes each
- Grace-grace node has twice 72 cores: 144 total per node.
- 600 Grace-Hopper nodes
  19 racks of 32 nodes
- GH200 Grace-Hopper superchip combines
  each containing one 72-core Grace CPU (120 GiB LPDDR5X)
  and H200 GPU (96 GiB HBM3)
- GH node: 34 Tflops of FP64
  1979 Tflops of FP16 for Machine Learning.

Grace Hopper superchip



Grace Hopper superchip

Grace-Hopper GPU/CPU Memory Coherence



single page table enabled by address translation services (**ATS**)

71

- Single page table
- CPU and GPU can access data by paging.

CUDA and others

- CUDA is for C and Fortran;
  limited to NVidia
- OpenACC, OpenCL: portable but tough to program
- AMD GPUs: 'HIP'
- Intel GPUs (hm): Sycl
- OpenMP offload: universal but maybe not highest performance
- Kokkos: portable to OpenMP and all GPUs

2. Kernels

Starting a kernel

Ordinary C/C++ function;

`__global__` means executable on device:

```
// hello.cu
__global__ void helloWorldKernel() {
  printf("Hello, World!\n");
}
```

Execute on device with blocksize and gridsize
(here both one, so one core):

```
helloWorldKernel<<<1, 1>>>();
```

Translation code to kernel

```
for ( int i=0; i<n; ++i ) {
  int threadIdx.x = i;
  f(i);
}

__global__ computef( /* fargs */ ) {
  int i = threadIdx.x;
  f(i);
}

int gridsize,blocksize;
computef<<<gridsize,blocksize>>>( fargs );
```

- Kernel is the body of the inner loop;
- Kernel is executed for each thread in the grid;
- Kernel consults `treadIdx` to find the 'iteration' number.
- Note: kernel invocation returns immediately, use `cudaDeviceSynchronize` to wait until device finishes

About threads

- CUDA creates thread blocks until all data has been processed
- You can tune the block size up to some limit;
- Make sure to create enough blocks.

Code:

```
// blocksize.cu
printf("Device Name: %s\n", prop.name);
printf("Max threads per block: %d\n",
       prop.maxThreadsPerBlock);
printf("Max block dimensions:\n %d x %d x %d\n",
       prop.maxThreadsDim[0],
       prop.maxThreadsDim[1],
       prop.maxThreadsDim[2]);
```

Output:

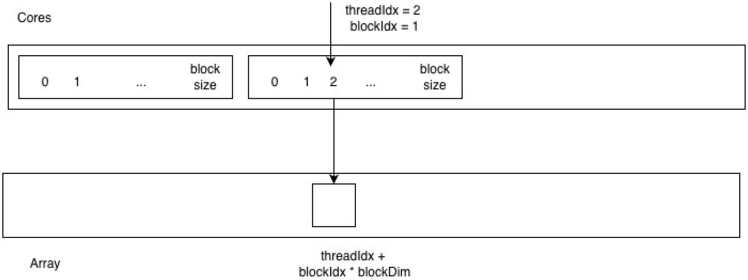
```
Device Name: NVIDIA GH200 120GB
Max threads per block: 1024
Max block dimensions:
 1024 x 1024 x 64
```

Thread indexing

```
int global_thread_id = blockIdx.x * blockDim.x + threadIdx.x; // or 2d, 3d variant
if ( global_thread_id >= datasize ) return;
```

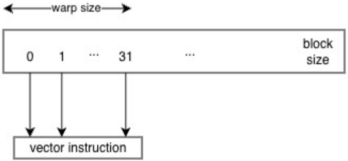
- Global thread index from block and thread coordinate;
- Global index often used as index in the data;
- You can have more threads than data;
  no problem but you do need to test.

Thread indexing illustrated



Thread indexing rationale

Why consecutive threads on consecutive array locations?
Why not let each thread have a consecutive chunk?



Data on the device

```
float
  *x, *y, *z;
size_t nbytes = N*sizeof(float);
CU_ASSERT( cudaMallocManaged( &x,nbytes ) );
CU_ASSERT( cudaMallocManaged( &y,nbytes ) );
CU_ASSERT( cudaMallocManaged( &z,nbytes ) );
```

```
// Run kernel on 1M elements on the CPU
int blocksize = 1024;
int gridsize = N/blocksize+1;
add1<<<gridsize,blocksize>>>(N,x,y,z);
// Wait for GPU to finish before accessing data on host
CU_ASSERT( cudaDeviceSynchronize() );
```

- Managed allocation is accessible on host and device
- Device is asynchronous, so synchronization needed

Utility macro

```
#define CU_ASSERT( code ) { \
  cudaError_t err = code ; \
  if (err!=cudaSuccess) { \
    printf("error <<%s>> on line %d\n",cudaGetErrorString(err),__LINE__); \
    return 1; } }
```

- CUDA routines can return error code
- Check on fatal errors.

Exercise 1

Write a CUDA kernel that adds two arrays into a third.

Using the above snippets, finish the code, test for correctness.

Two-dimensional grid

```
dim3 blocksize(16,16);
dim3 gridsize( rowsize/blocksize.x+1,colsize/blocksize.y+1 );
printf(" .. grid is (%d,%d) blocks of (%d,%d)\n",
         gridsize.x,gridsize.y,blocksize.x,blocksize.y);
```

- Two-dimensional data calls for two-dimensional thread organization (same for three of course)
- Specify grid/block size with `dim3` objects.

Two-dimensional indexing

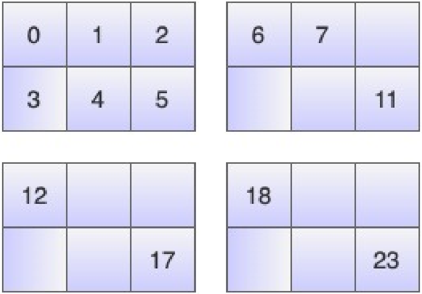
```
// index2d.cu
// Calculate the row and column indices for this thread
size_t row = blockIdx.y * blockDim.y + threadIdx.y;
size_t col = blockIdx.x * blockDim.x + threadIdx.x;

// Make sure we don't go out of bounds
if (row < height && col < width) {
// Calculate the linear index for the matrices
  size_t idx = row * width + col;

// Perform the addition
  d_c[idx] = d_a[idx] + d_b[idx];
```

- Compute row/col from x/y components
- Adjacent threads need to access adjacent data

Thread numbering



- Lexicographic ordering of blocks;
- Lexicographic ordering of threads in blocks.

Exercise 2

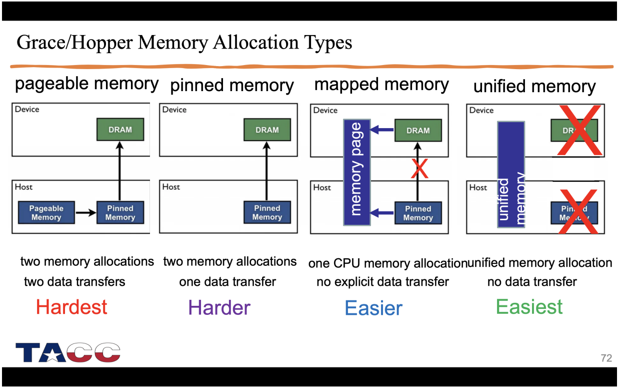
Create a two-dimensional array and let each thread write its global number in the elemenent where it is active:

```
output[output_loc] = global_thread_num;
```

Your program should recreate the layout of figure 28.

3. Memory

Host and device memory



- Host memory can be copied;
- can be visible on the device;
- can be unified with the device.

Explicit transfer

Allocate input data and copy host values:

```
// input allocated on device
int *dev_input;
cudaMalloc( &dev_input,bytesize );
// host values copied
cudaMemcpy( dev_input,input,bytesize,cudaMemcpyHostToDevice);
```

After the kernel execution copy output data back:

```
cudaDeviceSynchronize();
cudaMemcpy( output,dev_output,bytesize,cudaMemcpyDeviceToHost );
```

Free:

```
free(input); free(output);
cudaFree(dev_input); cudaFree(dev_output);
```

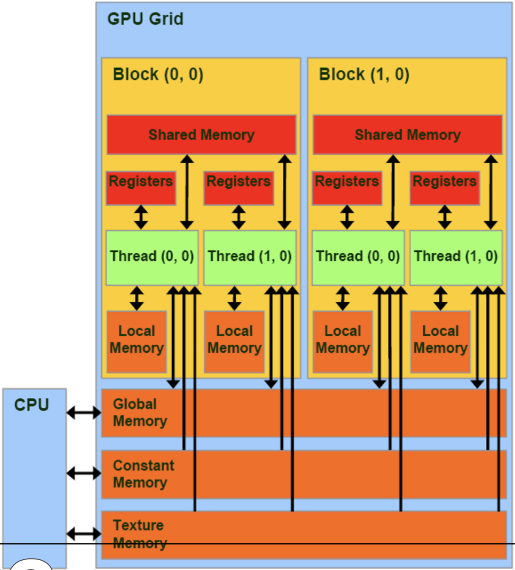
- Host holds pointer to device memory;
- used for copying in/out;
- device memory freed.

Implicit transfer

```
// add1m.cu
size_t nbytes = N*sizeof(float);
CU_ASSERT( cudaMallocManaged( &x,nbytes ) );
CU_ASSERT( cudaMallocManaged( &y,nbytes ) );
CU_ASSERT( cudaMallocManaged( &z,nbytes ) );
```

- Possibility one:
  allocate on the host with `cudaHostAlloc`
  using a flag of `cudaHostAllocMapped`.
  Pass `cudaHostGetDevicePointer` to the device.
- Easier: `cudaMallocManaged`;
  transfer is done through on-demand paging.
- (Impliciations for timing!)

GPU memory


GPU Grid
Block (0, 0)
Block (1, 0)
Shared Memory
Shared Memory
Registers
Registers
Registers
Registers
Thread (0, 0)
Thread (1, 0)
Thread (0, 0)
Thread (1, 0)
Local Memory
Local Memory
Local Memory
Local Memory
CPU
Global Memory
Constant Memory
Texture Memory

Shared memory

```
__shared__ float[32][32];
```

- Shared between threads in a block:
  managed cache per core
- Can be used as fast local storage
- Copy from RAM to ‘shared’, operate on shared, copy back.

4. Synchronization

Mechanisms

- Threads in a warp are always synchronized.
- Threads in a block can be synchronized with `__syncthreads`.
- Blocks in a grid can not be synchronized.
- Host and device can be synchronized with `cudaDeviceSynchronize`.

Warps and warp divergence

```
if ( threadIdx.%2==0 )
  functionA();
else
  functionB();
```

```
bool mask = (threadIdx.x%2==0)
if (mask) functionA();
if (not mask) functionB();
```

- Threads in a warp are synchronized
- Conditionals are serialized: *warp divergence*
- Try to rewrite your code…

Synchronization across blocks

- Threads blocks are not synchronized; can not be.
  Does this make sense from the original purpose of Graphics Processing Units (GPUs)?
- Solution 1: synchronize outside CUDA;
- Solution 2: use atomics;
- Solution 3: use a library that does it for you.

Example: sum reduction

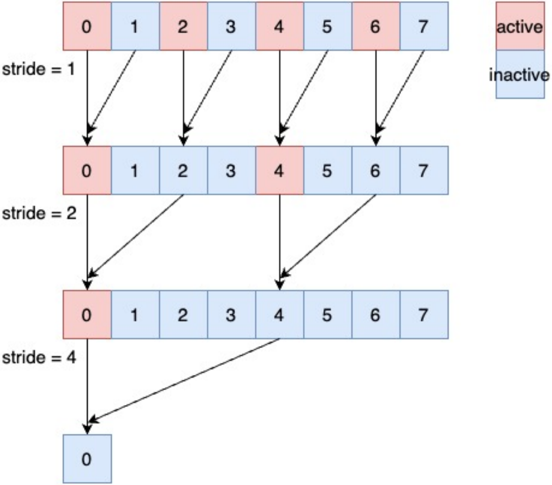
- Recursive summation in each block:

```
// redumy.cu
__global__
void reduce0(size_t n, float *input, float *y);
```

- Summing the blocks outside of CUDA:

```
for ( int b=0; b<gridsize; ++b)
  value += y[b*blocksize];
```

Implementation 1

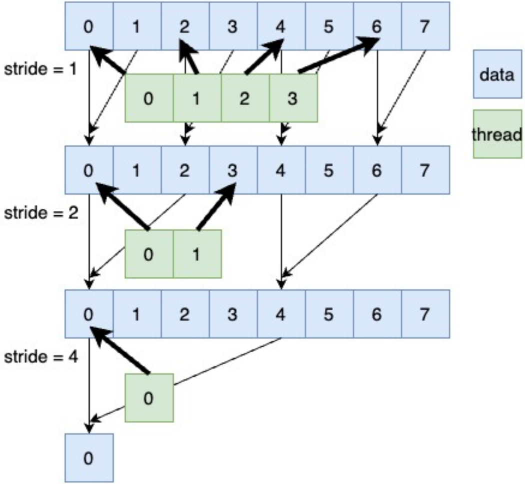


We let threads at distances $2, 4, 8, \ldots$ sum values from offsets $1, 2, 4, \ldots$.

Code

```
__global__
void reduce0(size_t n, float *input, float *y) {
  size_t
    thrd = threadIdx.x,
    global = blockDim.x*blockIdx.x + threadIdx.x;
  if (global>=n) return;
  y[global] = input[global];
  __syncthreads();
  for ( size_t offset=1; offset<=blockDim.x/2; offset*=2 ) {
    if (thrd%(2*offset)==0)
      y[global] += y[global+offset];
    __syncthreads();
  }
}
```

Implementation 2



- Non-obvious mapping thread $\leftrightarrow$ data

Code

```
// pointer to this block
float *data = y+blockDim.x*blockIdx.x;
for ( size_t offset=1; offset<=blockDim.x/2; offset*=2 ) {
  size_t index = 2 * offset * thrd;
  if (index+offset<blockDim.x)
    data[index] += data[index+offset];
  __syncthreads();
}
// we need the following statement really only for thread 0:
y[global] = data[thrd];
```

Implementation 3

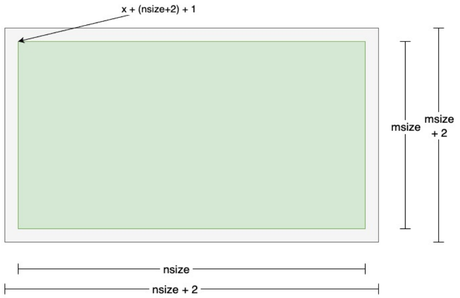
- Use 'Cuda UnBound'
- `https://github.com/NVIDIA/cccl/tree/main/cub`

```
// compute needed tmp space
cub::DeviceReduce::Reduce
  ( d_temp_storage, temp_storage_bytes,
    x, &d_nrm, num_items, devplus{}, 0.f );
// alloc tmp space
cudaMalloc(&d_temp_storage, temp_storage_bytes);
// actual reduction
cub::DeviceReduce::Reduce
  ( d_temp_storage, temp_storage_bytes,
    x, &d_nrm, num_items, devplus{}, 0.f );
CU_ASSERT( cudaDeviceSynchronize() );
cudaFree(&d_temp_storage);
```

5. Example: stencil

Five-point stencil

$$y_{ij} \leftarrow 4x_{ij} - x_{i-1,j} - x_{i+1,j} - x_{i,j-1} - x_{i,j+1}$$



- Streaming kernel, but rhs elements used multiple times
- We use bordered data for easy of programming.

Coordinates

```
// smooth2d.cu
auto global_row = [=] ( int64_t by,int64_t ty ) {
  return by * blockDim.y + ty; };
auto global_col = [=] ( int64_t bx,int64_t tx ) {
  return bx * blockDim.x + tx; };
auto global_loc = [nsize] ( int64_t row,int64_t col ) {
// one blank row, and each row is left & right bordered
  return (row+1) * (nsize+2) + col+1; };
```

```
int64_t bx = blockIdx.x, by = blockIdx.y, tx = threadIdx.x, ty = threadIdx.y;
int64_t my_global_row = global_row(by,ty);
int64_t my_global_col = global_col(bx,tx);
if ( my_global_row<0 or my_global_col<0
     or my_global_row>=msize or my_global_col>=nsize )
  return;
```

```
y[my_global_loc] = 4*x[my_global_loc]
  - x[ global_loc( my_global_row-1,my_global_col ) ]
  - x[ global_loc( my_global_row+1,my_global_col ) ]
  - x[ global_loc( my_global_row  ,my_global_col-1 ) ]
  - x[ global_loc( my_global_row  ,my_global_col+1 ) ];
```

- Utility functions,
- translation thread to global coordinate

Use of local memory

If elements read multiple times:

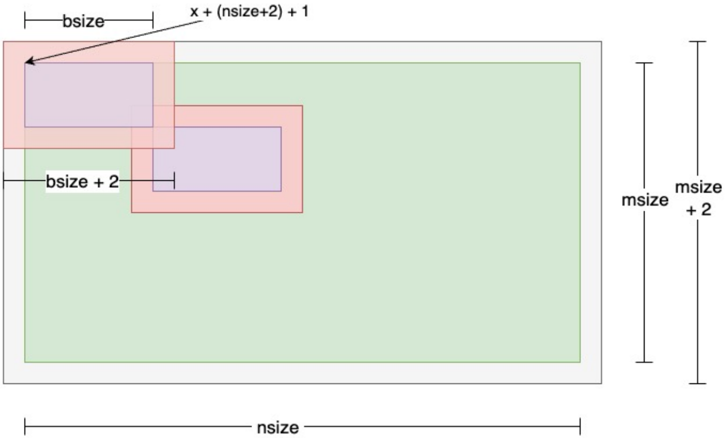
- Copy to local ('shared') data,
- Operate on that
- Write back

Shared memory

```
__shared__ float sums[BLOCKSIZE];
sums[thrd_idx] = x[global_loc];
__syncthreads();
```

- Shared memory size of thread block: static dimensions
- Shared between all threads of block

Shared memory in stencil computation



- Assume bordered global domain,
- Make local block bordered too
- Tricky programming!

Exercise 3

Take your 2D vector addition code;
use shared memory. Do you see a performance difference?
If not, can you find a way to make this visible?

For instance, try:

```
__global__ void add_with_coef( double *out,double *in,
    double *coef,int ncoeff) {
      for ( icoeff=0; icoeff<ncoeff; ++icoeff ) {
        for ( .... i ... )
          a[i] += coef[icoeff] * in[i];
      }
}
```

6. Timing, profiling and such

Device queries

```
// device.cu
int ndev;
auto status = cudaGetDeviceCount(&ndev);
```

```
cudaDeviceProp properties;
cudaGetDeviceProperties(&properties,idev);
```

```
cout << "Device " << idev << "=" << properties.name << '\n';
cout << "  async: " << properties.asyncEngineCount << '\n';
cout << "  unified: " << properties.unifiedAddressing << '\n';

cout << "  capability: " << properties.major << "." << properties.minor << '\n';
cout << "  multiprocs: " << properties.multiProcessorCount << '\n';
cout << "  clock rate: " << properties.clockRate << '\n';

cout << "  global memory: " << properties.totalGlobalMem << '\n';
cout << "  shared mem/block: " << properties.sharedMemPerBlock << '\n';
```

Block size

Code:

```
// blocksize.cu
printf("Device Name: %s\n", prop.name);
printf("Max threads per block: %d\n",
       prop.maxThreadsPerBlock);
printf("Max block dimensions:\n %d x %d x %d\n",
       prop.maxThreadsDim[0],
       prop.maxThreadsDim[1],
       prop.maxThreadsDim[2]);
```

Output:

```
Device Name: NVIDIA GH200 120GB
Max threads per block: 1024
Max block dimensions:
 1024 x 1024 x 64
```

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
float milliseconds = 0.0f;

cudaEventRecord(start);
cudaEventSynchronize(start);
// device operation goes here
cudaEventRecord(stop);
cudaEventSynchronize(stop);
cudaEventElapsedTime(&milliseconds, start, stop);
```

- You can time on the host, but make sure to synchronize
- Time on the device with events, act like barriers.

-

Exercise 4 `code`